

Gabarito · 10 testes unitários

Um teste para cada camada · resolvido, rodado e conferido — código, caminho completo do arquivo e saída do terminal

ALUNO(A)	Paulo Odilon
REPOSITÓRIO	github.com/podilon/petshop-x4
PASTA LOCAL	C:\Users\podil\PROJETOS\AULA-NEXT-JS\PROJETO_PETSHOP\petshop-x4
AULAS	x4trend.com/cursos/next/testes
SUÍTE FINAL	17 suites · 90 testes · todos passando (79 que já existiam + 11 novos)

Por que 11 testes e não 10? A tarefa 8 pede um arquivo novo com **dois** testes — o caminho feliz do formato brasileiro e o travessão da data inválida. As outras nove tarefas acrescentam um teste cada. Nenhum arquivo de produção foi alterado.

Índice das tarefas

#	TAREFA	ARQUIVO	NÍVEL
1	Nome com exatamente 2 letras é aceito	src/modules/pets/usecases/__tests__/criar-pet.usecase.test.ts	FÁCIL
2	Consulta com o id certo antes de apagar	src/modules/pets/usecases/__tests__/remover-pet.usecase.test.ts	FÁCIL
3	O banco é consultado uma vez só	src/modules/pets/usecases/__tests__/listar-pets.usecase.test.ts	FÁCIL
4	Nome de uma letra também é capitalizado	src/modules/pets/handlers/__tests__/criar-pet.handler.test.ts	FÁCIL
5	A data de cadastro aparece no card	src/components/features/petshop/__tests__/CardPet.test.tsx	FÁCIL
6	As quatro espécies estão no select	src/components/features/petshop/__tests__/FormAdicionarPet.test.tsx	MÉDIO
7	Repasse puro do handler	src/modules/pets/handlers/__tests__/remover-pet.handler.test.ts	MÉDIO
8	Data inválida vira travessão	src/shared/utils/__tests__/formatar-data.test.ts	MÉDIO
9	Lista vazia não é erro	src/hooks/pets/__tests__/use-pets.test.tsx	MÉDIO
10	A data também é formatada no buscarPorId	src/modules/pets/repositories/__tests__/repositorio-pet.test.ts	DESAFIO

Como ler cada tarefa

Cada bloco abaixo segue o formato de entrega do Manual do Aluno: primeiro **o código** — o bloco `test(...)` inteiro, com os comentários PREPARAR / AGIR / VERIFICAR e o caminho completo do arquivo em que ele mora — e depois **a saída do terminal** daquele arquivo rodando sozinho, com o `PASS` e o teste novo marcado com `✓`. Entre os dois, o

descritivo explica *por que* aquele caso de borda merece um teste e *qual decisão* de escrita foi tomada. Cada tarefa fecha com a prova do ciclo vermelho: o que quebra na produção para aquele teste falhar.

Nível fácil · tarefas 1 a 5

1 Nome com exatamente 2 letras é aceito

FÁCIL

Use case · regra de negócio

O QUE PROVA

Que `criarPetUseCase` aceita um nome de exatamente 2 letras e deixa o pet chegar ao repositório.

A regra de produção é `if (dados.nome.length < 2) throw`. Os testes que já existiam provam apenas o lado de fora da fronteira: um nome de **1 letra** é recusado. Ninguém provava o lado de dentro. Este teste fecha a fronteira pelo lado que passa: com `"Bo"`, nenhuma exceção é lançada e o objeto chega intacto ao `repositorioPet.salvar`.

1 · O CÓDIGO

CAMINHO COMPLETO

C:\Users\podil\PROJETOS\AULA-NEXT-JS\PROJETO PETSHOP\petshop-x4\src\modules\pets\usecases__tests__\criar-pet.usecase.test.ts

NO REPOSITÓRIO

src/modules/pets/usecases/__tests__/criar-pet.usecase.test.ts

```
// Tarefa 1 – Nome com exatamente 2 letras é aceito
test("deve aceitar nome com exatamente 2 letras", async () => {
  // PREPARAR
  const pet = { nome: "Bo", especie: "cachorro", dono: "Ana Silva" };

  // AGIR
  await criarPetUseCase(pet);

  // VERIFICAR
  expect(salvarMock).toHaveBeenCalledWith(pet);
});
```

Como o teste se encaixa no arquivo. O mock do repositório já estava montado no topo do arquivo (`const salvarMock = repositorioPet.salvar as jest.Mock`), e o `beforeEach` já devolve um pet pronto. Só foi preciso acrescentar o bloco `test(...)`.

2 · A SAÍDA DO TERMINAL

```
$ npx jest criar-pet.usecase --verbose
```

```
PASS src/modules/pets/usecases/__tests__/criar-pet.usecase.test.ts  
criarPetUseCase
```

- ✓ deve salvar o pet quando os dados forem válidos (4 ms)
- ✓ deve recusar quando nome, espécie ou dono estiverem ausentes (7 ms)
- ✓ deve recusar nome com menos de 2 letras (2 ms)
- ✓ deve recusar espécie fora da lista permitida (2 ms)
- ✓ deve recusar dono com menos de 3 caracteres (1 ms)
- ✓ NÃO deve tocar no banco quando os dados forem inválidos (2 ms)
- ✓ deve aceitar nome com exatamente 2 letras (1 ms)

```
Test Suites: 1 passed, 1 total
```

```
Tests: 7 passed, 7 total
```

```
Snapshots: 0 total
```

```
Time: 0.922 s, estimated 1 s
```

```
Ran all test suites matching criar-pet.usecase.
```

Detalhe que decide. Verificar `toHaveBeenCalledWith(pet)` — e não só "não lançou erro" — prova as duas coisas de uma vez: a validação deixou passar e o use case não mexeu nos dados antes de salvar.

Prova de que o teste morde (ciclo vermelho). Trocando `< 2` por `< 3` no use case, o teste falha com *Nome do pet deve ter ao menos 2 letras*.

2 Consulta com o id certo antes de apagar

FÁCIL

Use case · regra de negócio

O QUE PROVA

Que `removerPetUseCase(42)` procura no banco o pet 42 — e não outro número qualquer.

O use case faz duas coisas: consulta o pet e, se ele existir, remove. Os testes antigos usavam sempre `id = 1`, que é o mesmo valor do pet devolvido pelo `beforeEach` — ou seja, um id trocado passaria despercebido. Aqui o id pedido (`42`) é escolhido de propósito para ser diferente de tudo que existe no arquivo, e a verificação é sobre o **argumento** que chegou ao repositório.

1 · O CÓDIGO

CAMINHO COMPLETO

C:\Users\podil\PROJETOS\AULA-NEXT-JS\PROJETO PETSHOP\petshop-x4\src\modules\pets\usecases__tests__\remover-pet.usecase.test.ts

NO REPOSITÓRIO

src/modules/pets/usecases/__tests__/remover-pet.usecase.test.ts

```
// Tarefa 2 – Consulta com o id certo antes de apagar
test("deve consultar o pet pelo id recebido antes de remover", async () => {
  // PREPARAR – o mock precisa "achar" um pet, senão o use case lança erro
  buscarPorIdMock.mockResolvedValue({ id: 42, nome: "Rex" });

  // AGIR
  await removerPetUseCase(42);

  // VERIFICAR
  expect(buscarPorIdMock).toHaveBeenCalledWith(42);
});
```

Como o teste se encaixa no arquivo. O `beforeEach` já devolve um pet válido, mas o teste sobrescreve o mock com `mockResolvedValue({ id: 42, ... })`: sem um pet "encontrado", o use case lançaria *Pet não encontrado* e o teste morreria antes da verificação.

2 · A SAÍDA DO TERMINAL

```
$ npx jest remover-pet.usecase --verbose

PASS src/modules/pets/usecases/__tests__/remover-pet.usecase.test.ts
  removerPetUseCase
    ✓ deve remover o pet quando ele existir no banco (5 ms)
    ✓ deve recusar a remoção quando o pet não existir (4 ms)
    ✓ NÃO deve chamar remover quando o pet não existir (1 ms)
    ✓ deve consultar o pet pelo id recebido antes de remover (10 ms)

Test Suites: 1 passed, 1 total
Tests:       4 passed, 4 total
Snapshots:  0 total
Time:        0.916 s, estimated 1 s
Ran all test suites matching remover-pet.usecase.
```

Detalhe que decide. Cuidado com o tipo: `42` e `"42"` são coisas diferentes para o Jest. `toHaveBeenCalledWith(42)` só passa com o número.

Prova de que o teste morde (ciclo vermelho). Fixando `buscarPorId(1)` na produção, o teste falha apontando *Expected: 42 / Received: 1*.

3 O banco é consultado uma vez só

FÁCIL

Use case · regra de negócio

O QUE
PROVA

Que `listarPetsUseCase()` chama `buscarTodos` **exatamente uma vez**.

Os testes existentes provavam o *resultado* da listagem (a lista repassada, a lista vazia), mas nenhum olhava para o *custo* da operação. Uma consulta duplicada devolve exatamente o mesmo resultado — e passaria em todos os testes antigos — enquanto dobra o tempo e o preço de cada carregamento de tela. Este teste protege esse ponto cego.

1 · O CÓDIGO

CAMINHO COMPLETO

C:\Users\podil\PROJETOS\AULA-NEXT-JS\PROJETO PETSHOP\petshop-x4\src\modules\pets\usecases__tests__\listar-pets.usecase.test.ts

NO REPOSITÓRIO

src/modules/pets/usecases/__tests__/listar-pets.usecase.test.ts

```
// Tarefa 3 – O banco é consultado uma vez só
test("deve consultar o repositório exatamente uma vez", async () => {
  // PREPARAR
  buscarTodosMock.mockResolvedValue([]);

  // AGIR
  await listarPetsUseCase();

  // VERIFICAR
  expect(buscarTodosMock).toHaveBeenCalledTimes(1);
});
```

Como o teste se encaixa no arquivo. `toHaveBeenCalledTimes(1)` é o matcher que conta chamadas. O `jest.clearAllMocks()` do `beforeEach` é o que torna a contagem confiável: sem ele, as chamadas dos testes anteriores continuariam somando.

2 · A SAÍDA DO TERMINAL

```
$ npx jest listar-pets.usecase --verbose
```

```
PASS src/modules/pets/usecases/__tests__/listar-pets.usecase.test.ts
```

```
  listarPetsUseCase
```

```
    ✓ deve repassar a lista retornada pelo repositório (5 ms)
```

```
    ✓ deve devolver lista vazia quando não houver pets cadastrados (1 ms)
```

```
    ✓ deve consultar o repositório exatamente uma vez (1 ms)
```

```
Test Suites: 1 passed, 1 total
```

```
Tests:       3 passed, 3 total
```

```
Snapshots:   0 total
```

```
Time:        0.898 s, estimated 1 s
```

```
Ran all test suites matching listar-pets.usecase.
```

Detalhe que decide. `toHaveBeenCalled()` (sem o `Times`) passaria com 1, 2 ou 50 chamadas. É a diferença entre provar que aconteceu e provar quantas vezes aconteceu.

Prova de que o teste morde (ciclo vermelho). Duplicando o `await repositorioPet.buscarTodos()` na produção, o teste falha com *Expected number of calls: 1 / Received: 2*.

4 Nome de uma letra também é capitalizado

FÁCIL

Handler · normalização de entrada

O QUE PROVA

Que o handler transforma `"r"` em `"R"` sem quebrar — mesmo sendo uma letra só.

A função de produção é `texto.charAt(0).toUpperCase() + texto.slice(1).toLowerCase()`. Com um único caractere, `slice(1)` devolve string vazia — um caso de borda clássico, onde implementações ingênuas costumam devolver `undefined` ou lançar erro. Os testes antigos só usavam nomes de três ou mais letras.

1 · O CÓDIGO

CAMINHO COMPLETO

C:\Users\podil\PROJETOS\AULA-NEXT-JS\PROJETO PETSHOP\petshop-x4\src\modules\pets\handlers__tests__\criar-pet.handler.test.ts

NO REPOSITÓRIO

src/modules/pets/handlers/__tests__/criar-pet.handler.test.ts

```
// Tarefa 4 – Nome de uma letra também é capitalizado
test("deve capitalizar quando o nome tiver uma letra só", async () => {
  // AGIR
  await criarPetHandler({ nome: "r", especie: "cachorro", dono: "Ana Silva" });

  // VERIFICAR
  expect(useCaseMock).toHaveBeenCalledWith(
    expect.objectContaining({ nome: "R" })
  );
});
```

Como o teste se encaixa no arquivo. O `expect.objectContaining` é proposital: o handler monta um objeto com `nome`, `especie` e `dono`. Sem ele, seria preciso escrever os três campos — e o teste quebraria se alguém acrescentasse um quarto. O teste declara o que importa e ignora o resto.

2 · A SAÍDA DO TERMINAL

```
$ npx jest criar-pet.handler --verbose
```

```
PASS src/modules/pets/handlers/__tests__/criar-pet.handler.test.ts
```

```
criarPetHandler
```

- ✓ deve capitalizar o nome e remover os espaços laterais (4 ms)
- ✓ deve converter a espécie para minúsculas e remover espaços (2 ms)
- ✓ deve remover os espaços laterais do nome do dono (2 ms)
- ✓ deve incluir a raça quando ela vier preenchida (1 ms)
- ✓ NÃO deve incluir o campo raça quando ela vier vazia (1 ms)
- ✓ NÃO deve incluir o campo raça quando ela vier só com espaços (1 ms)
- ✓ deve repassar nome vazio quando o nome vier só com espaços (1 ms)
- ✓ deve chamar o use case exatamente uma vez (1 ms)
- ✓ deve capitalizar quando o nome tiver uma letra só (2 ms)

```
Test Suites: 1 passed, 1 total
```

```
Tests: 9 passed, 9 total
```

```
Snapshots: 0 total
```

```
Time: 0.91 s, estimated 1 s
```

```
Ran all test suites matching criar-pet.handler.
```

Detalhe que decide. Este teste vive na camada do handler, não do use case: aqui o `criarPetUseCase` está mockado, então nenhuma validação de negócio roda. O foco é só a normalização.

Prova de que o teste morde (ciclo vermelho). Trocando o guard por `if (texto.length < 2) return texto`, o teste falha: chega `"r"` em vez de `"R"`.

5 A data de cadastro aparece no card

FÁCIL

Componente · React puro

O QUE PROVA

Que o card mostra na tela a data que veio na prop `criadoEm`.

Aqui não existe mock nenhum: o `CardPet` só recebe props e devolve HTML. O teste usa a fábrica `criarPetFake` sobrescrevendo apenas `criadoEm` — uma data diferente da data padrão da fábrica (`10/01/2024`), justamente para provar que o valor **renderizado veio da prop** e não de um texto fixo no componente.

1 · O CÓDIGO

CAMINHO COMPLETO

C:\Users\podil\PROJETOS\AULA-NEXT-JS\PROJETO PETSHOP\petshop-x4\src\components\features\petshop__tests__\CardPet.test.tsx

NO REPOSITÓRIO

src/components/features/petshop/__tests__/CardPet.test.tsx

```
// Tarefa 5 – A data de cadastro aparece no card
test("deve exibir a data de cadastro do pet", () => {
  // PREPARAR + AGIR
  render(
    <CardPet pet={criarPetFake({ criadoEm: "25/12/2024" })} onRemove={jest.fn()} />
  );

  // VERIFICAR
  expect(screen.getByText(/25\/12\/2024/)).toBeInTheDocument();
});
```

Como o teste se encaixa no arquivo. No card a data está grudada em `"Dono: ... · Cadastrado em: ..."`, dentro do mesmo nó de texto. Por isso a busca usa **expressão regular** (entre barras) em vez de aspas — e cada barra da data precisa de `\` na frente.

2 · A SAÍDA DO TERMINAL

```
$ npx jest CardPet --verbose
```

```
PASS src/components/features/petshop/__tests__/CardPet.test.tsx
CardPet
  ✓ deve exibir nome, espécie, dono e data de cadastro do pet (40 ms)
  ✓ deve exibir a raça entre parênteses quando ela existir (7 ms)
  ✓ NÃO deve exibir parênteses com raça quando ela não existir (4 ms)
  ✓ deve avisar o pai com o id do pet ao clicar em Remove (103 ms)
  ✓ deve exibir a data de cadastro do pet (7 ms)
```

```
Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
Snapshots:   0 total
Time:        1.15 s
Ran all test suites matching CardPet.
```

Detalhe que decide. `getByText` com string exige que o texto do elemento case por inteiro; com regex, basta conter.

Prova de que o teste morde (ciclo vermelho). Removendo `Cadastrado em: {pet.criadoEm}` do componente, o teste falha com *Unable to find an element with the text: /25/12/2024/*.

6 As quatro espécies estão no select

MÉDIO

Componente · React puro

**O QUE
PROVA**Que o formulário oferece **exatamente** as quatro espécies que o use case aceita.

Existem duas listas de espécies no projeto: `ESPECIES` no formulário e `ESPECIES_VALIDAS` no `criar-pet.usecase.ts`. Elas moram em camadas diferentes e nada impede que saiam de sincronia. Se o formulário oferecesse uma espécie a mais, o usuário só descobriria o erro depois de preencher tudo e apertar enviar. Este teste amarra a tela ao domínio.

1 · O CÓDIGO**CAMINHO COMPLETO**

```
C:\Users\podil\PROJETOS\AULA-NEXT-JS\PROJETO PETSHOP\petshop-x4\src\components\features\petshop\__tests__\FormAdicionarPet.test.tsx
```

NO REPOSITÓRIO

```
src/components/features/petshop/__tests__/FormAdicionarPet.test.tsx
```

```
// Tarefa 6 – As quatro espécies estão no select
test("deve oferecer exatamente as quatro espécies válidas", () => {
  // PREPARAR + AGIR
  render(<FormAdicionarPet onSubmit={jest.fn()} />);

  // VERIFICAR
  const opcoes = screen.getAllByRole("option") as HTMLOptionElement[];

  expect(opcoes).toHaveLength(4);
  expect(opcoes.map((o) => o.value)).toEqual([
    "cachorro",
    "gato",
    "coelho",
    "hamster",
  ]);
});
```

Como o teste se encaixa no arquivo. A verificação usa `o.value`, **não o texto**: na tela aparece "Cachorro" com maiúscula, mas o `value` é `"cachorro"` — e é o `value` que trafega para o use case. O `toEqual` com array também trava a **ordem**.

```
$ npx jest FormAdicionarPet --verbose
```

```
PASS src/components/features/petshop/__tests__/FormAdicionarPet.test.tsx
FormAdicionarPet
  ✓ deve permitir digitar nos campos de texto (262 ms)
  ✓ deve permitir selecionar a espécie (45 ms)
  ✓ deve chamar onSubmit com o payload correto quando o formulário for enviado (185 ms)
  ✓ NÃO deve incluir raca no payload quando o campo vier vazio (110 ms)
  ✓ deve limpar os campos após o envio (164 ms)
  ✓ deve desabilitar o botão e mostrar Salvando... quando salvando for true (9 ms)
  ✓ deve oferecer exatamente as quatro espécies válidas (15 ms)
```

```
Test Suites: 1 passed, 1 total
Tests:       7 passed, 7 total
Snapshots:   0 total
Time:        1.846 s, estimated 2 s
Ran all test suites matching FormAdicionarPet.
```

Detalhe que decide. `toHaveLength(4)` sozinho não bastaria (quatro espécies erradas passariam); o `toEqual` sozinho já garante o tamanho, mas a asserção explícita documenta a intenção.

Prova de que o teste morde (ciclo vermelho). Removendo `"hamster"` da constante `ESPECIES`, o teste falha em `Expected length: 4 / Received length: 3`.

7 Repasse puro do handler

MÉDIO

Handler · porta de entrada do domínio

O QUE
PROVA

Que `removePetHandler` repassa o id ao use case **sem alterar nada** e chamando **uma vez só**.

Diferente do `criarPetHandler`, este handler não normaliza nada — ele é uma porta. Testar um repasse parece trivial, mas é exatamente onde regressões silenciosas nascem: um `parseInt` acrescentado "por segurança", uma chamada duplicada dentro de um `retry`. As duas asserções juntas travam identidade e cardinalidade.

1 · O CÓDIGO

CAMINHO COMPLETO

```
C:\Users\podil\PROJETOS\AULA-NEXT-JS\PROJETO PETSHOP\petshop-x4\src\modules\pets\handlers\__tests__\remove-pet.handler.test.ts
```

NO REPOSITÓRIO

```
src/modules/pets/handlers/__tests__/remove-pet.handler.test.ts
```

```
// Tarefa 7 – Repasse puro do handler
test("deve repassar o id ao use case exatamente uma vez", async () => {
  // AGIR
  await removePetHandler(7);

  // VERIFICAR
  expect(useCaseMock).toHaveBeenCalledWith(7);
  expect(useCaseMock).toHaveBeenCalledTimes(1);
});
```

Como o teste se encaixa no arquivo. O ponto delicado é o `jest.mock` com **função de fábrica** (a forma longa, `() => ({ ... })`). Sem ela o Jest carrega o arquivo real do use case, que puxa o repositório e o banco — e o teste morre com `DATABASE_URL não foi configurada` antes de rodar.

2 · A SAÍDA DO TERMINAL

```
$ npx jest remover-pet.handler --verbose

PASS src/modules/pets/handlers/__tests__/remover-pet.handler.test.ts
  removerPetHandler
    ✓ deve repassar o id para o use case sem alterar (5 ms)
    ✓ deve repassar o id ao use case exatamente uma vez (2 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:  0 total
Time:        0.959 s, estimated 1 s
Ran all test suites matching remover-pet.handler.
```

Detalhe que decide. O arquivo indicado pela tarefa já existia no repositório com um teste de repasse; o teste desta tarefa foi acrescentado ao mesmo arquivo, mantendo o mock e o `beforeEach` que já estavam montados.

Prova de que o teste morde (ciclo vermelho). Duplicando `await removerPetUseCase(id)` na produção, o teste falha com *Expected number of calls: 1 / Received: 2*.

8 Data inválida vira travessão

Utilitário · função pura

MÉDIO

ARQUIVO NOVO

O QUE PROVA Que `formatarDataCriacao` devolve `"-"` com uma data inválida, em vez de mostrar `Invalid Date` ao usuário.

Arquivo novo, e o mais simples de todos: função pura, nada mockado. São dois testes — o caminho feliz (formato brasileiro) e o caminho de erro (travessão). O segundo é o que interessa: sem esse guard, um valor corrompido no banco chegaria à tela como `Invalid Date`. Vale notar que os outros arquivos de teste **mockam** este utilitário; este é o único lugar do projeto onde a função real roda de verdade.

1 · O CÓDIGO

CAMINHO COMPLETO

C:\Users\podil\PROJETOS\AULA-NEXT-JS\PROJETO PETSHOP\petshop-x4\src\shared\utils__tests__\formatar-data.test.ts

NO REPOSITÓRIO

src/shared/utils/__tests__/formatar-data.test.ts

```
/**
 * Testes do formatarDataCriacao – função pura, nada é mockado.
 */
import { formatarDataCriacao } from "../formatar-data";

describe("formatarDataCriacao", () => {
  test("deve formatar a data no padrão brasileiro", () => {
    // PREPARAR – mês 0 = janeiro! 0 12 é a hora, evita erro de fuso
    const data = new Date(2024, 0, 10, 12);

    // AGIR
    const resultado = formatarDataCriacao(data);

    // VERIFICAR
    expect(resultado).toBe("10/01/2024, 12:00");
  });

  test("deve devolver travessão quando a data for inválida", () => {
    const resultado = formatarDataCriacao("banana");

    expect(resultado).toBe("-");
  });
});
```

Como o teste se encaixa no arquivo. Duas armadilhas. 1) Em JavaScript o mês começa em 0: `new Date(2024, 0, 10)` é 10 de janeiro. 2) O travessão `-` é diferente do hífen `-` — o caractere foi copiado direto de `formatar-data.ts`.

2 · A SAÍDA DO TERMINAL

```
$ npx jest formatar-data --verbose
```

```
PASS src/shared/utils/__tests__/formatar-data.test.ts
  formatarDataCriacao
    ✓ deve formatar a data no padrão brasileiro (34 ms)
    ✓ deve devolver travessão quando a data for inválida (1 ms)
```

```
Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        0.996 s, estimated 1 s
Ran all test suites matching formatar-data.
```

Detalhe que decide. A hora `12` não é decorativa: o formatador fixa `timeZone: "America/Fortaleza"`, e um horário no meio do dia impede que a conversão de fuso empurre a data para o dia anterior ou seguinte.

Prova de que o teste morde (ciclo vermelho). Trocando `return "-"` por `return "-"`, o teste falha com *Expected: "-" / Received: "-"*.

9 Lista vazia não é erro

MÉDIO

Hook · estado da tela

O QUE PROVA

Que, quando a API devolve [], o hook sai do carregamento e **não** mostra mensagem de erro.

Sistema recém-instalado não pode parecer quebrado. O hook tem três estados (`carregando`, `erro`, `pets`) e a lista vazia é o caso em que é fácil escorregar: basta alguém tratar "sem resultados" como falha para o usuário novo ver um alerta vermelho na primeira visita. O teste prova as duas metades: o carregamento termina e nenhum alerta aparece.

1 · O CÓDIGO

CAMINHO COMPLETO

C:\Users\podil\PROJETOS\AULA-NEXT-JS\PROJETO PETSHOP\petshop-x4\src\hooks\pets__tests__\use-pets.test.tsx

NO REPOSITÓRIO

src/hooks/pets/__tests__/use-pets.test.tsx

```
// Tarefa 9 – Lista vazia não é erro
test("deve exibir lista vazia sem erro quando a API não devolver pets", async () => {
  // PREPARAR
  listarMock.mockResolvedValue([]);

  // AGIR
  render(<Cobaia />);

  // VERIFICAR – espera sair do "Carregando..."
  await waitFor(() => {
    expect(screen.queryByText("Carregando pets...")).not.toBeInTheDocument();
  });

  // e agora: o alerta de erro NÃO pode estar na tela
  expect(screen.queryByRole("alert")).not.toBeInTheDocument();
});
```

Como o teste se encaixa no arquivo. Regra de ouro: para testar que algo **não está** na tela, use `queryBy...`. O `getBy...` lança erro quando não encontra — e quebraria justamente no caso que se quer provar. O componente-cobaia `<Cobaia />` já existia no topo do arquivo.

2 · A SAÍDA DO TERMINAL

```
$ npx jest use-pets --verbose
```

```
PASS src/hooks/pets/__tests__/use-pets.test.tsx
  usePets
    ✓ deve exibir carregando no estado inicial (32 ms)
    ✓ deve listar os pets quando a busca der certo (23 ms)
    ✓ deve exibir a mensagem de erro quando listPetsAction falhar (69 ms)
    ✓ deve usar "Erro desconhecido" quando o valor lançado não for um Error (42 ms)
    ✓ deve atualizar a lista após adicionarPet (58 ms)
    ✓ deve exibir salvando durante o adicionarPet (48 ms)
    ✓ deve atualizar a lista após removerPet (26 ms)
    ✓ NÃO deve marcar salvando durante removerPet (29 ms)
    ✓ deve exibir a mensagem de erro quando adicionarPet falhar (32 ms)
    ✓ deve usar "Erro desconhecido" quando adicionarPet falhar com valor não-Error (26 ms)
    ✓ deve exibir a mensagem de erro quando removerPet falhar (24 ms)
    ✓ deve usar "Erro desconhecido" quando removerPet falhar com valor não-Error (22 ms)
    ✓ deve exibir lista vazia sem erro quando a API não devolver pets (7 ms)
```

```
Test Suites: 1 passed, 1 total
Tests:       13 passed, 13 total
Snapshots:   0 total
Time:        1.449 s
Ran all test suites matching use-pets.
```

Detalhe que decide. O `await waitFor` vem primeiro por um motivo: enquanto `carregando` é true, a cabaia renderiza só o texto de carregamento e **nenhum** alerta existiria de qualquer forma. Sem essa espera, o teste passaria sem ter verificado nada.

Prova de que o teste morde (ciclo vermelho). Fazendo o hook chamar `setErro("Nenhum pet encontrado")` quando a lista vem vazia, o teste falha ao encontrar o elemento com `role="alert"`.

Desafio · tarefa 10

10 A data também é formatada no buscarPorId

DESAFIO

Repositório · Drizzle + PostgreSQL

O QUE PROVA

Que `buscarPorId` — e não só `buscarTodos` — formata a data antes de devolver o pet.

O repositório tem um `mapearPet` compartilhado, e os testes antigos só exercitavam esse mapeamento pelo caminho do `buscarTodos`. Se alguém otimizasse o `buscarPorId` devolvendo a linha crua do banco, a suíte inteira continuaria verde e a tela passaria a mostrar um objeto `Date` cru no lugar da data formatada.

1 · O CÓDIGO

CAMINHO COMPLETO

C:\Users\podil\PROJETOS\AULA-NEXT-JS\PROJETO PETSHOP\petshop-x4\src\modules\pets\repositories__tests__\repositorio-pet.test.ts

NO REPOSITÓRIO

src/modules/pets/repositories/__tests__/repositorio-pet.test.ts

```
// Tarefa 10 (desafio) – A data também é formatada no buscarPorId
test("buscarPorId deve formatar a data antes de devolver o DT0", async () => {
  // PREPARAR – uma linha crua, como viria do banco
  const linha = {
    id: 1,
    nome: "Rex",
    especie: "cachorro",
    dono: "Ana",
    raca: null,
    criadoEm: new Date(),
  };

  // a escada: select() devolve algo com from(), que devolve algo com where()
  selectMock.mockReturnValue({
    from: jest.fn().mockReturnValue({
      where: jest.fn().mockResolvedValue([linha]),
    }),
  });

  // AGIR
  const pet = await repositorioPet.buscarPorId(1);

  // VERIFICAR – o mock do formatar-data devolve sempre a mesma string
  expect(pet?.criadoEm).toBe("10/01/2024");
});
```

Como o teste se encaixa no arquivo. O desafio é o **mock encadeado**. O Drizzle usa métodos em escada: `db.select().from(tabela).where(condição)`. O mock precisa imitar essa escada degrau por degrau — `select()` devolve um objeto com `from`, que devolve um objeto com `where`, que finalmente resolve a lista de linhas. Um degrau faltando e o erro é `Cannot read properties of undefined`.

2 · A SAÍDA DO TERMINAL

```
$ npx jest repositorio-pet --verbose

PASS src/modules/pets/repositories/__tests__/repositorio-pet.test.ts
  repositorioPet
    ✓ buscarPorId deve formatar a data antes de devolver o DTO
    buscarTodos
      ✓ deve mapear as linhas do banco para o DTO (4 ms)
      ✓ deve omitir raca no DTO quando a coluna vier nula (2 ms)
      ✓ deve devolver lista vazia quando não houver pets no banco (2 ms)
      ✓ deve repassar raca com espaços quando a coluna vier preenchida assim (R19) (1 ms)
    buscarPorId
      ✓ deve devolver o pet mapeado quando encontrar o id (2 ms)
      ✓ deve devolver null quando o id não existir no banco (2 ms)
    salvar
      ✓ deve enviar nome, especie, dono e raca ao values do insert (1 ms)
      ✓ deve devolver o pet mapeado após o insert (1 ms)
      ✓ deve lançar erro quando o insert não retornar nenhuma linha (23 ms)
    remover
      ✓ deve chamar delete com where filtrando pelo id (1 ms)

Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        1.108 s
Ran all test suites matching repositorio-pet.
```

Detalhe que decide. Por que `new Date()` sem argumentos pode aqui? Porque o `formatar-data` está mockado no topo do arquivo e devolve sempre `"10/01/2024"`, ignorando a data recebida. É a exceção que confirma a regra da tarefa 8 — e é justamente por isso que a asserção prova que o formatador **foi chamado**, não o que ele calcula.

Prova de que o teste morde (ciclo vermelho). Fazendo `buscarPorId` devolver a linha crua (`{ ...pet }`) em vez de `mapearPet(pet)`, o teste falha: `criadoEm` chega como `Date`, não como `"10/01/2024"`.

Suíte completa

Os 79 testes que já existiam continuam verdes junto com os 11 novos. Nenhum arquivo de produção foi tocado — o [git status](#) mostra apenas os nove arquivos de teste modificados mais o arquivo novo da tarefa 8.

```
$ npm test

PASS src/components/features/petshop/__tests__/FormAdicionarPet.test.tsx
PASS src/hooks/pets/__tests__/use-pets.test.tsx
PASS src/components/features/petshop/__tests__/CardPet.test.tsx
PASS src/modules/pets/repositories/__tests__/repositorio-pet.test.ts
PASS src/app/api/pets/__tests__/route.test.ts
PASS src/shared/utils/__tests__/formatar-data.test.ts
PASS src/modules/pets/usecases/__tests__/remover-pet.usecase.test.ts
PASS src/modules/pets/usecases/__tests__/listar-pets.usecase.test.ts
PASS src/modules/pets/handlers/__tests__/criar-pet.handler.test.ts
PASS src/modules/pets/usecases/__tests__/criar-pet.usecase.test.ts
PASS src/modules/pets/handlers/__tests__/remover-pet.handler.test.ts
PASS src/app/api/pets/__tests__/id-route.test.ts
PASS src/actions/pets/__tests__/delete-pet.action.test.ts
PASS src/actions/pets/__tests__/create-pet.action.test.ts
PASS src/actions/pets/__tests__/list-pets.action.test.ts
PASS src/modules/pets/handlers/__tests__/listar-pets.handler.test.ts
PASS src/shared/__tests__/ambiente.test.ts

Test Suites: 17 passed, 17 total
Tests:       90 passed, 90 total
Snapshots:   0 total
Time:        4.438 s, estimated 7 s
Ran all test suites.
```

Verificação: todo teste foi visto falhar

Um teste que passa desde o primeiro segundo pode não estar verificando nada. Para cada uma das dez tarefas, o código de produção foi quebrado de propósito, o teste foi visto **falhar**, e a produção foi restaurada em seguida. A quebra concreta de cada um está descrita no rodapé vermelho da tarefa correspondente.

#	TAREFA	PRODUÇÃO CERTA	PRODUÇÃO QUEBRADA
1	Nome com exatamente 2 letras é aceito	verde ✓	vermelho ✗
2	Consulta com o id certo antes de apagar	verde ✓	vermelho ✗
3	O banco é consultado uma vez só	verde ✓	vermelho ✗
4	Nome de uma letra também é capitalizado	verde ✓	vermelho ✗
5	A data de cadastro aparece no card	verde ✓	vermelho ✗
6	As quatro espécies estão no select	verde ✓	vermelho ✗
7	Repasse puro do handler	verde ✓	vermelho ✗
8	Data inválida vira travessão	verde ✓	vermelho ✗
9	Lista vazia não é erro	verde ✓	vermelho ✗
10	A data também é formatada no buscarPorId	verde ✓	vermelho ✗

Depois da verificação, tudo voltou ao lugar. A suíte final acima (90 testes) foi rodada com todos os arquivos de produção restaurados ao estado original do repositório.

Observação sobre fuso horário (tarefa 8)

O único teste da entrega que depende do relógio da máquina. O `formatarDataCriacao` fixa `timeZone: "America/Fortaleza"`, mas `new Date(2024, 0, 10, 12)` é construída no fuso **local** de quem roda. Em qualquer máquina no horário de Brasília (UTC-3, o caso desta entrega) o resultado é `"10/01/2024, 12:00"`, como o enunciado espera. Numa máquina em UTC — um servidor de CI, por exemplo — o mesmo código produziria `"10/01/2024, 09:00"` e o teste falharia. Se um dia esta suíte for para uma esteira de CI, a versão à prova de fuso é construir a data já em UTC: `new Date(Date.UTC(2024, 0, 10, 15))`, que dá 12:00 em Fortaleza venha de onde vier.

Cobertura

A cobertura já era 100% antes desta tarefa e continua 100% depois — e é exatamente esse o ponto. Cobertura mede **o que foi executado**, não o que foi garantido. Todas as linhas exercitadas pelos onze testes novos já eram percorridas por algum teste antigo; o que não existia era alguém *verificando o comportamento* naquelas linhas. Os números não mudam. A confiança, sim.

```
$ npm run test:cov
-----|-----|-----|-----|-----|-----
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files |    100 |     100 |     100 |     100 |
-----|-----|-----|-----|-----|-----
Test Suites: 17 passed, 17 total
Tests:       90 passed, 90 total
Snapshots:   0 total
Time:        5.303 s
Ran all test suites.
```

Checklist de entrega

SITUAÇÃO	ITEM
OK ✓	<code>npm test</code> roda tudo verde — os 79 antigos mais os 11 novos
OK ✓	Cada teste está no arquivo indicado pela tarefa
OK ✓	Todos os nomes seguem o padrão "deve ... quando ...", em português
OK ✓	Os comentários PREPARAR / AGIR / VERIFICAR estão visíveis
OK ✓	Nenhum arquivo de produção foi alterado
OK ✓	Este documento traz o código e a saída do terminal das 10 tarefas
OK ✓	Cada teste foi visto falhar antes de passar
OK ✓	O arquivo de aquecimento não foi criado (não faz parte da entrega)

Gabarito funcional · Curso **Testes na Arquitetura X4** · github.com/podilon/petshop-x4 · x4trend.com/cursos/next/testes